Software OFDM Modem: Architecture and Analysis of the amodem Library

Kristopher B. Kirby, KE4AHR
kris.kirby@gmail.com

***Abstract*—This paper presents a comprehensive technical analysis of the amodem software library, an open-source Python implementation of an Orthogonal Frequency-Division Multiplexing (OFDM) modem designed for audio-channel communication. The system enables data transmission between computers using audio signals, supporting both air-gapped communication via speakers and microphones, and direct audio cable connections. We examine the complete signal processing pipeline, analyze the DSP and filter functions for potential generation of unintended out-of-band noise that may impact adjacent carriers, and provide recommendations for spectral efficiency improvements. An appendix provides detailed data flow tracing through each function and module.**

***Index Terms*—Audio communication, digital signal processing, OFDM, QAM modulation, software-defined modem.**

I. INTRODUCTION

ORTHOGONAL Frequency-Division Multiplexing (OFDM) has become the dominant modulation technique in modern wireless and wired communication systems, from Wi-Fi (IEEE 802.11) to LTE and 5G cellular networks. The amodem library, developed by Roman Zeyde and hosted at github.com/romanz/amodem, provides an accessible Python implementation of OFDM principles for audio-frequency communication.

The system operates within the audio frequency band (typically 1–11 kHz), achieving bitrates from 1 kbps to 80 kbps depending on channel conditions. The software modulates binary data into audio signals transmitted through standard sound cards, enabling communication without network connectivity. This is particularly valuable for air-gapped security scenarios.

Table I summarizes the key system parameters. The modem supports multiple configurations with varying trade-offs between throughput and robustness.

Parameter	Value	Description
Sampling Rate	8/16/32 kHz	Based on bitrate mode
Baud Rate	1000 sym/s	Fixed symbol rate
Modulation	BPSK–256-QAM	Adaptive constellation
Carriers	1–10	2–11 kHz range
Max Bitrate	80 kbps	256-QAM, 10 carriers
Frame Size	250 bytes	CRC-32 protected

TABLE I: System Parameters

II. SYSTEM ARCHITECTURE

The amodem library follows a modular architecture with clear separation between signal processing, framing, and I/O operations. The codebase consists of approximately 15 Python modules organized into functional layers: entry point and CLI handling, high-level control flow, signal processing, DSP core functions, and infrastructure.

Module	Function
__main__.py	CLI, argument parsing, compression
main.py	Send/receive orchestration
send.py	OFDM modulation, carrier generation
recv.py	OFDM demodulation, bit recovery
detect.py	Carrier detection, synchronization
dsp.py	FIR filters, MODEM, Demux classes
framing.py	Data framing, CRC-32 checksums
sampling.py	Polyphase interpolation
equalizer.py	Adaptive channel equalization
config.py	Configuration parameters

TABLE II: Core Modules

III. TRANSMITTER PROCESSING

The transmission process converts binary input data into a modulated audio signal through a series of well-defined stages, as illustrated in Fig. 1.

Fig. 1. Transmitter signal processing chain.

A. Data Framing

Input data is segmented into 250-byte blocks, each protected by a CRC-32 checksum. The Framer class prepends a 1-byte length field and 4-byte CRC to each payload. The BitPacker class converts each byte into 8 bits (LSB first) for modulation. Fig. 2 shows the frame structure.

Fig. 2. Data frame structure (255 bytes maximum).

B. Symbol Mapping

The MODEM class implements configurable QAM constellation mapping. Bits are grouped according to constellation size (1 bit for BPSK, 2 for QPSK, 4 for 16-QAM, 6 for 64-QAM, 8 for 256-QAM) and mapped to complex symbols normalized to unit maximum magnitude. Fig. 3 shows representative constellations.

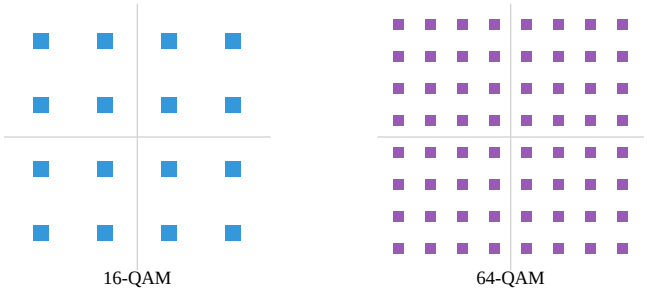

Fig. 3. QAM constellation diagrams.

C. OFDM Modulation

The Sender class performs OFDM modulation by multiplying QAM symbols with pre-computed carrier waveforms. For each 1 ms symbol period, N_{freq} symbols are transmitted simultaneously on orthogonal subcarriers spaced at 1 kHz intervals. The modulated signal is the dot product of the symbol vector with the carrier matrix, normalized by N_{freq}. Fig. 4 illustrates the resulting spectrum.

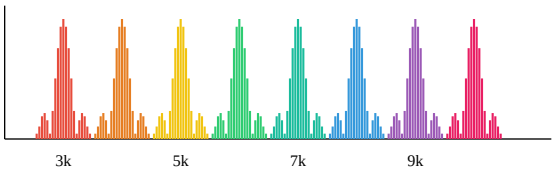

Fig. 4. OFDM subcarrier spectrum (8 carriers, 3–10 kHz).

D. Training Sequence

Before data transmission, a training sequence is sent: (1) 200 carrier-tone symbols for detection and synchronization, (2) 50 symbols of silence, (3) 200 QPSK training symbols generated by a PRBS (polynomial 0x1100b) for channel estimation, and (4) 50 symbols of silence. This enables carrier detection, frequency offset estimation, and adaptive equalizer training.

IV. RECEIVER PROCESSING

The receiver performs inverse operations with additional processing for synchronization and equalization, as shown in Fig. 5.

Fig. 5. Receiver signal processing chain.

A. Carrier Detection

The Detector class implements carrier detection using coherence measurement. For each symbol-length buffer, the coherence coefficient is computed as the normalized correlation with a complex exponential at the carrier frequency. Carrier presence is confirmed when coherence exceeds 0.9 for 90% of the expected duration (≈180 consecutive symbols).

B. Sampling and Interpolation

The Sampler class implements a polyphase interpolator for timing recovery. A bank of 1024 filters is pre-computed, each representing a different fractional sample delay. The interpolation kernel is a windowed

sinc function with raised cosine window (256 taps), providing smooth frequency response and minimal aliasing. The interpolator dynamically adjusts fractional delay based on estimated frequency error.

C. OFDM Demodulation

The Demux class performs OFDM demodulation using a DFT-based approach. Received samples are multiplied by conjugate complex exponentials at each subcarrier frequency and integrated over the symbol duration. The demodulation filters include proper normalization accounting for energy splitting between positive and negative frequency components.

D. Channel Equalization

Adaptive equalization compensates for channel distortion. The equalizer is trained using the known training sequence by solving a Wiener-Hopf equation via the Levinson-Durbin algorithm. A 10-tap FIR filter with 10 samples lookahead provides approximately 0.6 ms equalization span at 32 kHz sampling rate.

V. DSP AND NOISE ANALYSIS

A critical concern in multi-carrier systems is the potential for signal processing operations to generate out-of-band emissions that interfere with adjacent carriers.

A. Sources of Spectral Leakage

The primary source of unintended spectral content is the implicit rectangular windowing of each symbol. Symbols are modulated onto carriers using direct multiplication without pulse shaping. The rectangular pulse has a sinc-shaped spectrum with sidelobes decaying as 1/f, causing inter-carrier interference (ICI). Additional noise sources include interpolation filter artifacts from finite filter length and 16-bit quantization noise reduced by multi-carrier PAPR.

Source	Mechanism	Impact	Mitigation
Rect. pulse	Sinc sidelobes	-13 dB	Pulse shaping
Timing jitter	ICI	-20 dB	Timing recovery
Interpolation	Truncation	-40 dB	Longer filter
Quantization	16-bit limit	-81 dB	Dithering

TABLE III: Noise Sources and Mitigation

B. FIR Filter Implementation

The FIR filter implementation uses direct-form convolution with state preservation. All computations use NumPy 64-bit floating point with optimized BLAS routines, providing excellent numerical precision and minimizing accumulation error.

VI. RECOMMENDATIONS

A. Spectral Efficiency

Implementing raised-cosine pulse shaping with roll-off factor 0.25–0.35 would significantly reduce spectral sidelobes. Adding a cyclic prefix (10–20% of symbol duration) would improve robustness to multipath reflections in acoustic environments.

B. Enhanced Synchronization

Dedicating subcarriers to continuous pilot tones would enable tracking of time-varying channel conditions. Replacing correlation-based detection with Schmidl-Cox synchronization would provide more robust timing and frequency estimation in low-SNR conditions.

C. Error Correction

Adding convolutional coding (rate 1/2, K=7) or LDPC codes would improve performance in noisy channels. Symbol and bit interleaving across carriers and time would distribute burst errors, improving FEC effectiveness.

VII. CONCLUSION

The amodem library represents an elegant implementation of software-defined OFDM principles in Python. The modular architecture facilitates understanding and modification, making it valuable for education and practical air-gapped communication. Analysis reveals that rectangular symbol pulses and timing synchronization errors are primary sources of spectral leakage, acceptable for the target application but improvable with pulse shaping and enhanced synchronization techniques.

APPENDIX

DATA FLOW SUMMARY

A. Transmission Pipeline

Stage	Module	Function	I/O
1	__main__	_main()	CLI → Config
2	framing	Framer.encode()	Bytes → Frames
3	framing	BitPacker	Bytes → Bits
4	dsp	MODEM.encode()	Bits → Symbols
5	send	Sender.modulate()	Symbols → Audio

TABLE IV: Transmission Data Flow

B. Reception Pipeline

Stage	Module	Function	I/O
1	detect	Detector.run()	Audio → Sync
2	sampling	Sampler	Samples → Interp
3	dsp	Demux	Samples → Symbols
4	equalizer	train()	Training → Coeffs
5	dsp	MODEM.decode()	Symbols → Bits
6	framing	decode_frames()	Bits → Bytes

TABLE V: Reception Data Flow

REFERENCES

[1] R. Zeyde, "amodem: Audio MODEM Communication Library," GitHub, 2015. [Online]. Available: https://github.com/romanz/amodem

[2] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York, NY, USA: McGraw-Hill, 2008.

[3] R. van Nee and R. Prasad, *OFDM for Wireless Multimedia Communications*. Boston, MA, USA: Artech House, 2000.

[4] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978.

[5] N. Levinson, "The Wiener RMS error criterion in filter design and prediction," *J. Math. Phys.*, vol. 25, pp. 261–278, 1946.